We've replaced the `get 'store/index'` line with a call to define a root path, and in the process we added an `as: 'store_index'` option. The latter tells Rails to create `store_index_path` and `store_index_url` accessor methods, enabling existing code—and tests!—to continue to work correctly. Let's try it. Point a browser at http://localhost:3000/, and up pops our web page. See the following screenshot.



It might not make us rich, but at least we know everything is wired together correctly. It even tells us where to find the template file that draws this page.

Let's start by displaying a list of all the products in our database. We know that eventually we'll have to be more sophisticated, breaking them into categories, but this'll get us going.

We need to get the list of products out of the database and make it available to the code in the view that'll display the table. This means we have to change the `index` method in `store_controller.rb`. We want to program at a decent level of abstraction, so let's assume we can ask the model for a list of the products:

rails7/depot_d/app/controllers/store_controller.rb

```
  class StoreController < ApplicationController
    def index
»     @products = Product.order(:title)
    end
  end
```

We asked our customer if she had a preference regarding the order things should be listed in, and we jointly decided to see what happens if we display the products in alphabetical order. We do this by adding an `order(:title)` call to the `Product` model.

Now we need to write our view template. To do this, edit the `index.html.erb` file in `app/views/store`. (Remember that the path name to the view is built from the name